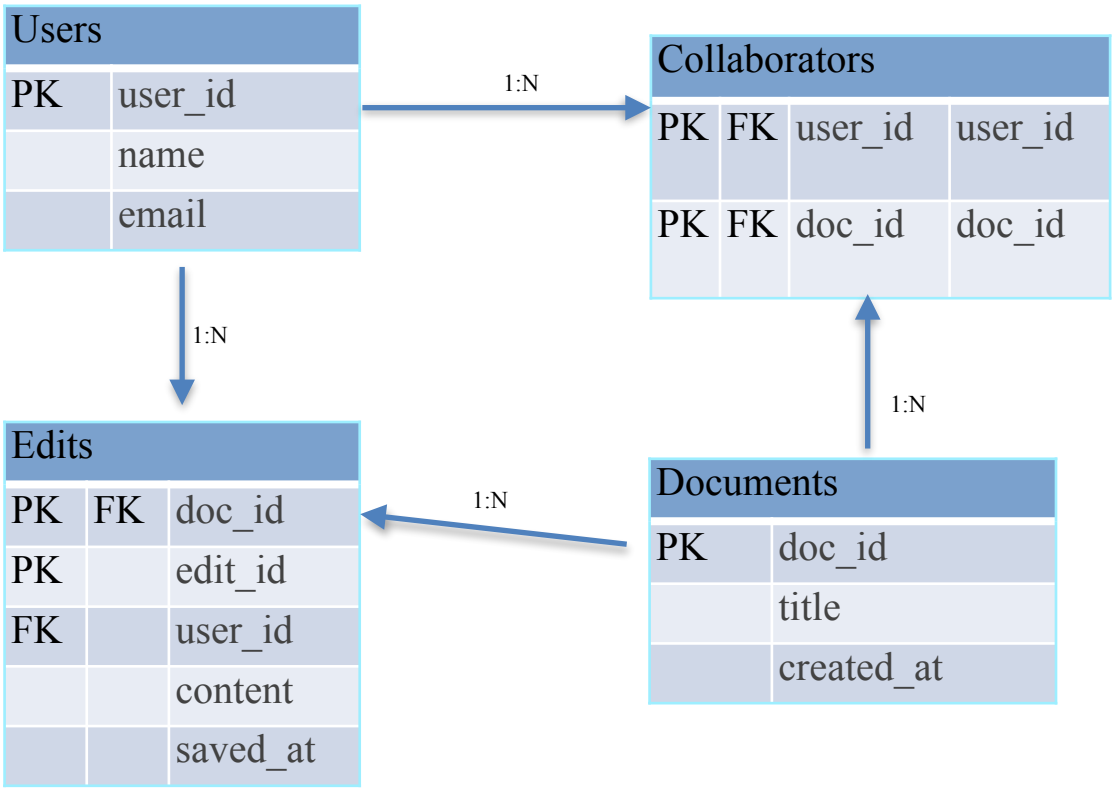
COMP-548DL
Big Data Management and Processing
Homework 2

Student Name: Farzana Asadi
Student ID: U254N2745

Exercise 1: Relational Schema



PK = primary key. FK = foreign key. Each user and document can have zero or more collaborator rows and zero or more edits. Each collaborator row and edit references exactly one user and one document.

Collaborators has the composite primary key (doc_id, user_id). The two columns reference Documents and Users. Edits has the composite primary key (doc_id, edit_id). Its doc_id references Documents, and user_id references Users.

Only current collaborators may save. Within a transaction, check the author's membership, assign an edit_id unique within that document, and insert the edit. Serialise saves and membership changes for the same document. Removing a collaborator deletes their membership row. Earlier edits still reference the author's record in Users.

Exercise 2: Performance Inefficiency Analysis

Retrieve all edits of a specific document in chronological order along with the editor's name and email.

```
SELECT
    e.edit_id, e.content, e.saved_at, u.user_id, u.name, u.email
FROM Edits e
JOIN Users u ON e.user_id = u.user_id
WHERE e.doc_id = :target_doc_id
ORDER BY e.saved_at ASC;
```

The query first selects the edits for :target_doc_id, then joins those edits to Users to obtain each author's name and email. For a document with a long history, a nested-loop execution plan may repeatedly look up the same authors. Other plans may build and probe a hash table. An index on (doc_id, saved_at) can help retrieve the matching edits in timestamp order. The cost of the join and any additional sorting depends on the execution plan, available indexes, and cache.

Exercise 3 MongoDB Data Model and Analysis

Selected database model

MongoDB is a document Store. The model uses two collections, users and documents. The users collection stores user profiles. Each document in the documents collection stores its metadata, current collaborator IDs, and append-only edit history.

MongoDB's _id represents user_id in users and doc_id in documents. Each embedded edit keeps the same fields returned by the SQL query in Exercise 2, edit_id, content, saved_at, user_id, name, and email. An edit_id is unique within its document.

Example records

The collaborators array stores the IDs of users who can edit the document. When a user saves, the application checks that their ID is in this array before adding the edit. Removing a collaborator removes their ID from the array, while their earlier edits stay in the history. To find one edit, the application uses the document's _id and the edit's edit_id.

Users collection

```
{
  _id: "usr_501",
  name: "Farzana",
  email: "farzana@homework.test"
}
```

Documents collection

```
{
 _id: "doc_1001",
 title: "Project notes",
 created_at: ISODate("2026-11-05T12:00:00Z"),
 collaborators: ["usr_501", "usr_502"],
 edits: [
  {
   edit_id: "edit_001",
   content: "Define the project goals.\n",
   saved_at: ISODate("2026-11-05T12:05:00Z"),
   user_id: "usr_501",
   name: "Farzana",
   email: “farzana@homework.test”
  },
  {
   edit_id: "edit_002",
   content: "Review the data sources.\n",
   saved_at: ISODate("2026-11-05T12:10:00Z"),
   user_id: "usr_502",
   name: "Ali",
   email: “ali@homework.test”
  }
 ]
}
```

How MongoDB Improves the Edit History Query

In Exercise 2, the SQL query joins Edits with Users to get each author’s name and email. In MongoDB, I copy name and email into each edit when it is saved. To get the full history, I read the document’s edits array by its _id and sort the results by saved_at. This returns the same fields as the SQL query without the author join. The trade-off is that names and emails are repeated, and the full history must fit within MongoDB’s document size limit.

Exercise 4: Implementation

GitHub Repository: https://github.com/Dawlatshahi/comp548dl-homework-2

The implementation details, Docker setup instructions, and verified query execution output are available in the repository.